May 29, 2026 | Branch `mym/coupled_cable` | Prepared by Mike Yan Michelis

RL Training Throughput Scaling

Isaac-Lift-Cable-Franka-v0 & Isaac-Lift-CablePlug-Franka-v0

Executive summary

Cable env. Total throughput scales with parallelism up to **4,096 envs (23,917 steps/s)**, plateaus there, and *regresses* above 4,096 envs — **6,144 envs is the first operating point where adding more environments yields fewer samples per second**. Below 4,096 envs the GPU still has headroom; above it, iteration time grows linearly with `num_envs` (≈ 1.03 ms/env) but throughput no longer climbs.

CablePlug env. Scales *poorly*: total throughput **peaks at only ∼20 steps/s near 8–16 envs** and then declines. Iteration time grows roughly $\mathcal{O}(N^{1.6-2})$ in `num_envs`; the simulator collection phase — not the PPO update — is responsible for $> 99\%$ of the cost.

Proxy vs. ADMM coupling (Cable env). ADMM on the plain cable scene is ∼10× slower per iter than proxy at 8 envs, growing to ∼53× at 32 envs; peak throughput stays ≤33 sps regardless of `num_envs`. The ADMM scaling on the cable scene closely mirrors the CablePlug scaling, suggesting the ADMM solver itself is the dominant cost in CablePlug.

1 Methodology

Each data point is one training run launched with

```
./isaaclab.sh train -rl_library rsl_rl -task <TASK>
                    -num_envs N -max_iterations 20 -headless
```

on a single NVIDIA RTX A6000 (49 GiB VRAM) in headless mode. The PPO trainer (RSL-RL) uses `num_steps_per_env = 24`; the simulator uses `decimation = 1`, `sim.dt = 1/60 s`. The first three iterations of every run are dropped as warm-up; reported numbers are the mean over the remaining iterations. "Steps per second" is the value RSL-RL prints to stdout, i.e. num_envs × num_steps_per_env/collection_time.

The metrics reported per run are:

- **Iteration time** (s) — wall-clock per RL iteration.
- **Steps per second** — environment steps produced per wall-clock second.
- **Startup time** (s) — time before the first training iteration starts producing samples.
- **Peak GPU memory** (MiB) — training memory delta over the system baseline.

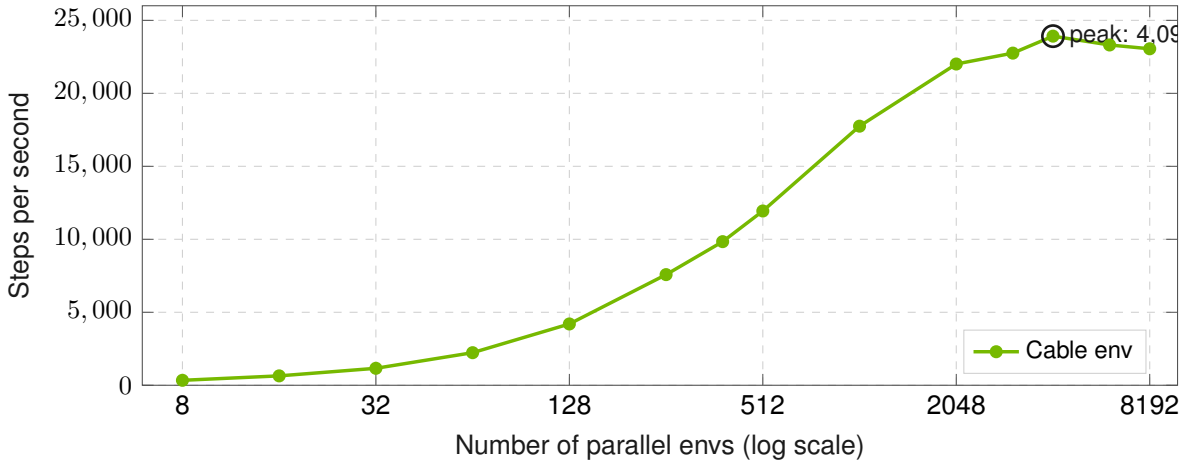
2 Isaac-Lift-Cable-Franka-v0

2.1 Scaling table

Cable env training throughput vs. number of parallel environments.

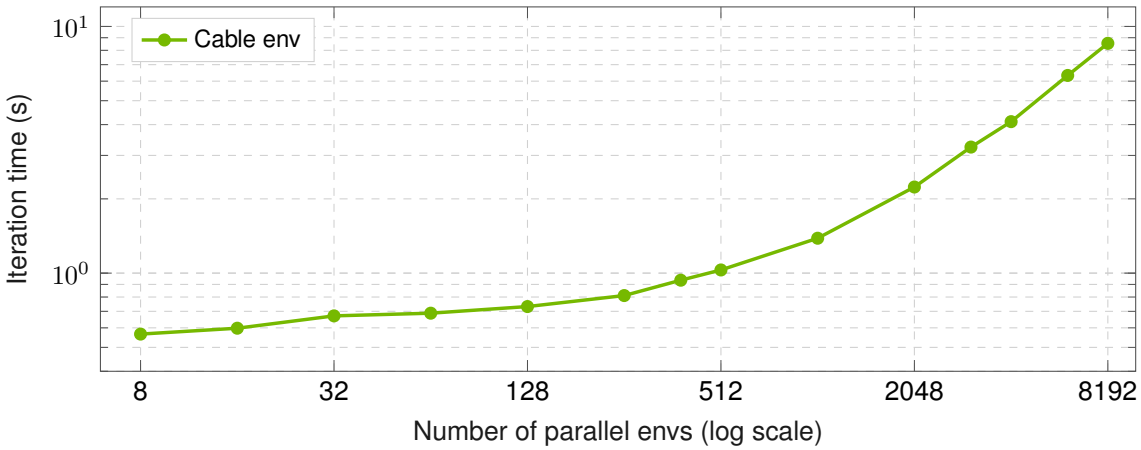
num_envs	Iter time (s)	Steps / s	Steps / s / env
8	0.57	339	42.4
16	0.60	643	40.2
32	0.67	1,162	36.3
64	0.69	2,232	34.9
128	0.73	4,199	32.8
256	0.81	7,580	29.6
384	0.94	9,839	25.6
512	1.03	11,939	23.3
1,024	1.39	17,750	17.3
2,048	2.23	22,008	10.8
3,072	3.24	22,755	7.4
4,096	**4.11**	**23,917**	**5.8**
6,144	6.33	23,313	3.8
8,192	8.54	23,052	2.8

2.2 Throughput vs. number of envs



Total throughput rises monotonically up to 4,096 envs, plateaus, then declines.

2.3 Iteration time vs. number of envs



Iter time is nearly flat below 512 envs (kernel-launch dominated), then grows linearly above ∼2,048 envs (≈1.03 ms/env) — the GPU-saturation signature.

2.4 When does adding more envs become a bad idea?

- **8 → 2,048 envs** — strong scaling. Every doubling of `num_envs` yields ≥24% more throughput.
- **2,048 → 4,096 envs** — diminishing returns. Each +1,024 envs adds only +3–5% throughput.
- **4,096 envs** — *throughput peak* (23,917 sps).
- **≥6,144 envs** — net negative. Throughput is lower than at 4,096, iter time is longer, per-env efficiency is worse. Do not operate here.

3 Proxy vs. ADMM coupling on the Cable env

The Cable env above uses **proxy** coupling (`CoupledProxySolverCfg`): the articulated robots are simulated in MJWarp while cables are in VBD; the gripper fingers are exposed as virtual proxies so VBD detects them as contacts on the cable. The cable plug env, by contrast, uses **ADMM** coupling (`CoupledAdmmSolverCfg`), which solves coupled mujoco-vbd constraints via an iterative alternating-direction scheme. ADMM is more rigorous but more expensive per step.

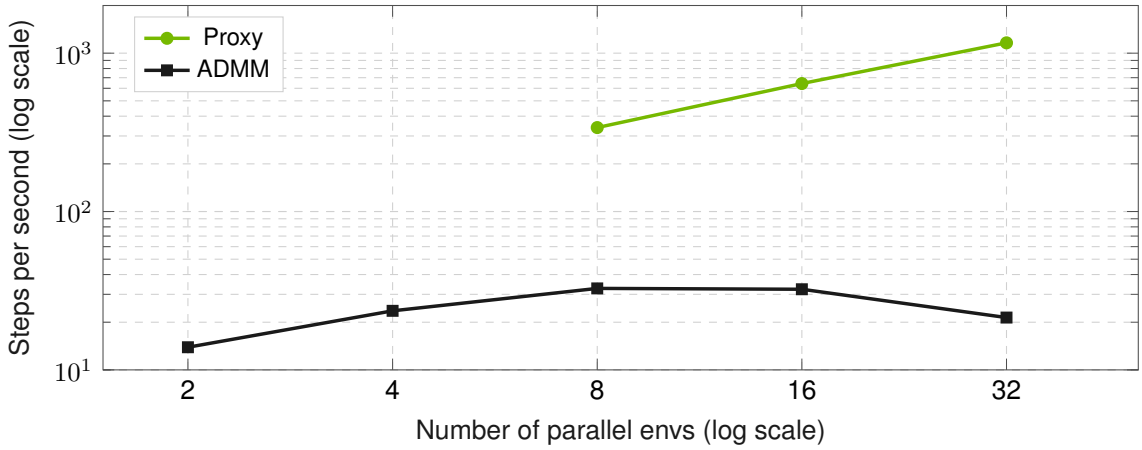
To compare the two on the same scene, the Cable env was re-instantiated with the ADMM solver and the ADMM recipe used by the cable plug env (iterations = 5, $\rho = 30$, $\gamma = 0.1$, baumgarte = 0.005, contact distance = 0.003, VBD $\beta = 10^3$, `num_substeps = 10`, contact stiffness $k_e = 10^5$). Task registered as `Isaac-Lift-Cable-ADMM-Franka-v0`.

3.1 Side-by-side scaling

Cable env throughput with proxy vs. ADMM coupling.

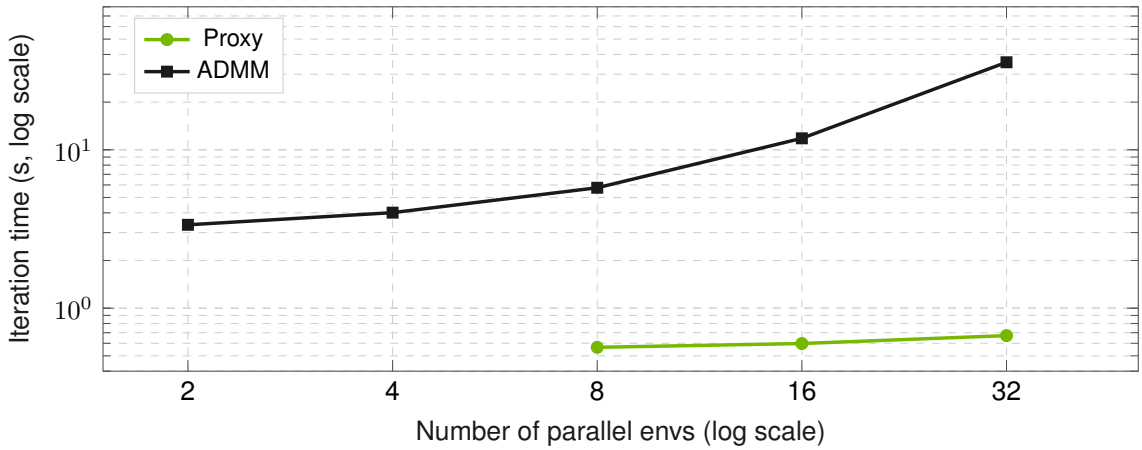
	Proxy		ADMM	
num_envs	Iter (s)	Steps / s	Iter (s)	Steps / s
2	—	—	3.36	13.9
4	—	—	4.01	23.6
8	0.57	339	5.76	32.7
16	0.60	643	11.82	32.3
32	0.67	1,162	35.71	21.4

3.2 Throughput vs. number of envs



Proxy throughput grows linearly with `num_envs`; ADMM throughput plateaus near 32 sps at 8–16 envs and then degrades. At 32 envs the proxy variant produces $\sim$54$\times$ more samples per second.

3.3 Iteration time vs. number of envs



Proxy iter time is flat at $\sim$0.6 s across this range; ADMM iter time grows roughly linearly above 8 envs and is already 53$\times$ proxy at 32 envs.

3.4 Interpretation

- **Per-iteration overhead.** At 8 envs, ADMM is $\sim$10$\times$ slower than proxy (5.76 s vs. 0.57 s); at 32 envs the gap widens to $\sim$53$\times$ (35.71 s vs. 0.67 s). Proxy iter time barely moves with `num_envs` in this range, while ADMM iter time scales super-linearly above 8 envs.
- **Throughput.** Proxy throughput rises linearly with envs in this range (339 $\rightarrow$ 1,162 sps for 8 $\rightarrow$ 32 envs); ADMM peaks at $\sim$33 sps near 8–16 envs and *drops* to 21 sps at 32 envs.
- **Why use ADMM at all?** ADMM is required by the cable plug env because the plug couples back into the cable through rigid–soft attachments that proxy coupling cannot represent. For the plain cable env where the gripper is the only rigid contact, proxy coupling is the correct choice: same physical scene, $\geq$50$\times$ better throughput, no stability issues observed.
- **Implication for the cable plug env.** The super-linear cost growth observed earlier in the CablePlug env ($\mathcal{O}(N^{1.6-2})$) is consistent with ADMM's scaling on the plain cable scene, which suggests the ADMM solver itself — not the plug geometry — is the dominant source of cost in the cable plug benchmarks.

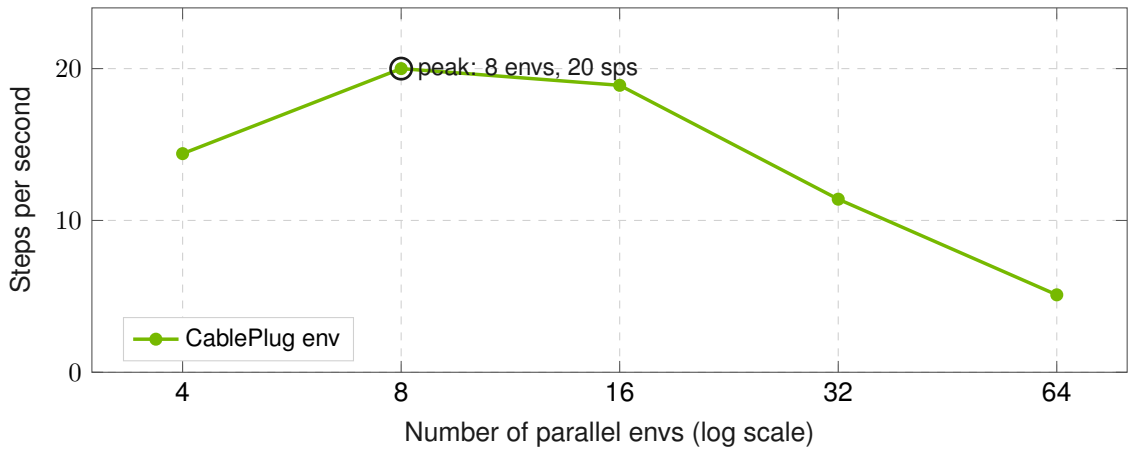
4 Isaac-Lift-CablePlug-Franka-v0

4.1 Scaling table

CablePlug env training throughput vs. number of parallel environments.

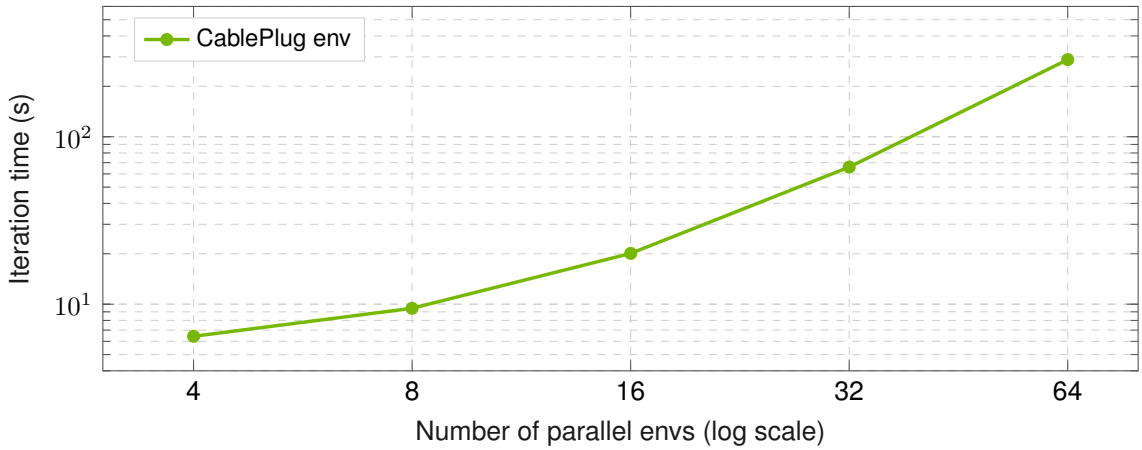
num_envs	Iter time (s)	Steps / s	Steps / s / env
4	6.43	14.4	3.59
8	**9.45**	**20.0**	**2.50**
16	20.10	18.9	1.18
32	65.87	11.4	0.36
64	288.88	5.1	0.08

4.2 Throughput vs. number of envs



Throughput peaks near 8–16 envs and falls thereafter — the opposite of healthy parallel scaling.

4.3 Iteration time vs. number of envs

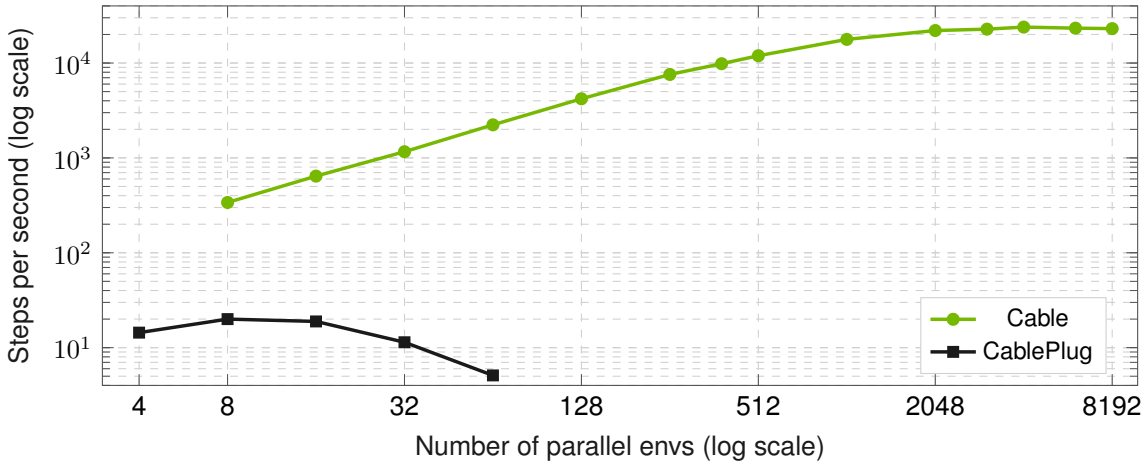


Iter time grows roughly $\mathcal{O}(N^{1.6-2})$ in `num_envs`: doubling envs costs 3–4$\times$ more time per iteration.

4.4 Implication

For CablePlug, more parallel envs is actively harmful past $\sim$16. The super-linear cost growth means the simulator does *not* parallelize the plug interactions efficiently; root-cause analysis of the collection phase is needed before this task can use more envs productively.

5 Head-to-head



Cable env achieves over 1,000$\times$ higher peak throughput than CablePlug env, and continues to scale well past 1,000 envs while CablePlug regresses past 16 envs.

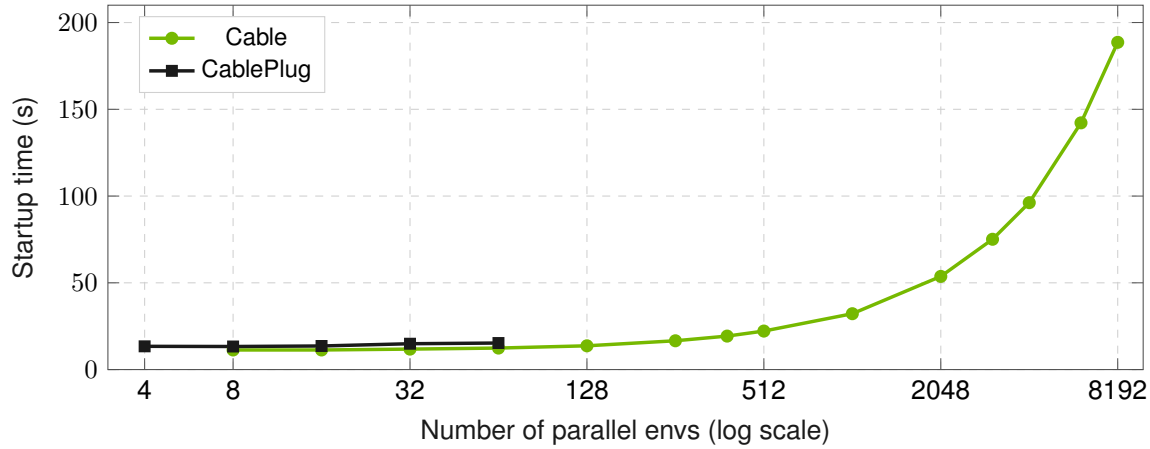
6 Startup cost

In addition to steady-state iteration time, every training run incurs a one-time startup cost (Python imports, Isaac Sim app init, scene cloning, USD stage build, RSL-RL setup) before the first training iteration produces samples. At very large env counts this is no longer negligible.

6.1 Startup time vs. number of envs

Startup time per run, by task and number of parallel envs.

num_envs	Cable startup (s)	CablePlug startup (s)
4	—	13.4
8	11.3	13.3
16	11.3	13.6
32	11.8	14.9
64	12.4	15.3
128	13.7	—
256	16.6	—
384	19.3	—
512	22.2	—
1,024	32.2	—
2,048	53.7	—
3,072	75.1	—
4,096	96.2	—
6,144	142.2	—
8,192	188.6	—



Cable env startup grows roughly linearly with `num_envs` above ∼512 envs (≈22 ms per added env). CablePlug startup is essentially flat (12–15 s) regardless of size — the cable plug's expense is at simulation time, not at scene-clone time.

6.2 Implication

For long training runs (many thousands of iterations) the startup cost is negligible. For shorter experiments at very large env counts it dominates:

- Cable @ 8,192 envs: startup is 188.6 s — more time than the first 22 training iterations combined.
- Cable @ 4,096 envs: startup of 96.2 s is roughly equal to the cost of 23 steady-state iterations.
- Cable @ 1,024 envs: startup of 32.2 s is roughly equal to 23 iterations.

- Cable @ $\leq$256 envs: startup is essentially fixed at 11–17 s.

A useful rule of thumb on this branch and GPU: startup amortizes after roughly 20–25 steady-state iterations, regardless of `num_envs`. Below that, smaller env counts have lower total wall-clock cost.

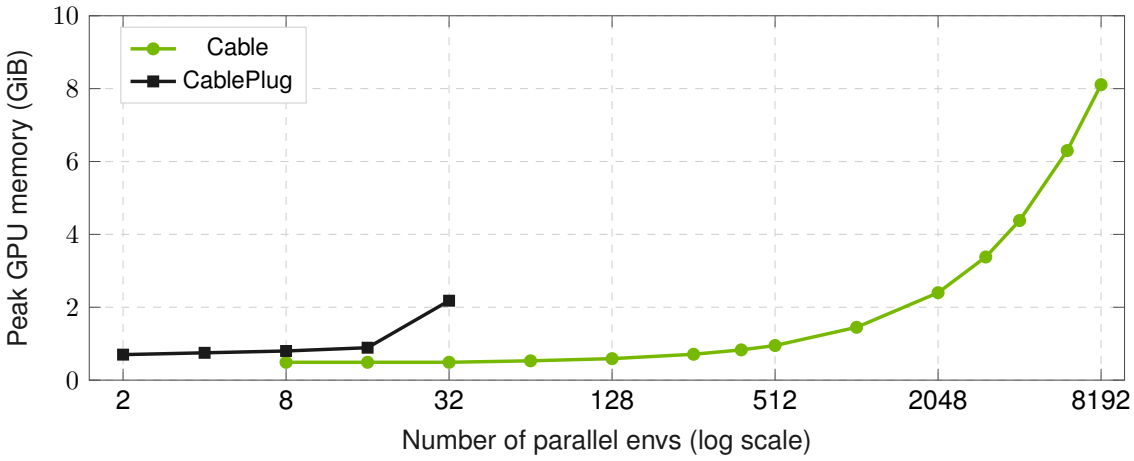
7 GPU memory usage

Peak GPU memory consumed by training, measured by polling `nvidia-smi -query-gpu=memory.used` at 0.5 s intervals during a short run (`-max_iterations 2`) and subtracting the pre-launch system baseline. The benchmark GPU is an NVIDIA RTX A6000 with **49,140 MiB (48 GiB)** of VRAM.

7.1 Memory vs. number of envs

Peak training GPU memory by task and number of parallel envs.

num_envs	Cable peak (GiB)	CablePlug peak (GiB)
2	—	0.70
4	—	0.75
8	0.49	0.80
16	0.49	0.89
32	0.49	2.18
64	0.53	—
128	0.59	—
256	0.71	—
384	0.83	—
512	0.95	—
1,024	1.45	—
2,048	2.40	—
3,072	3.38	—
4,096	4.38	—
6,144	6.30	—
8,192	8.11	—



Cable memory: fixed ∼0.5 GiB baseline for ≤32 envs, then grows linearly at ≈1 MiB per env. CablePlug memory: small (≤1 GiB) below 32 envs, then a sharp step to 2.2 GiB at 32 envs. The 48 GiB A6000 budget is far above either trace.

7.2 Implication

- Cable env memory is dominated by a fixed ∼0.5 GiB baseline (network weights, sim app, kit overhead) for env counts up to about 32, then grows linearly with `num_envs` (≈1 MiB per env) once cable storage dominates.
- At 8,192 envs Cable uses only 8.1 GiB — about 17% of A6000 VRAM. **Memory is not the bottleneck** for Cable's throughput plateau at 4,096 envs; that plateau is computational.
- CablePlug shows a sharp memory jump from 0.89 GiB (16 envs) to 2.18 GiB (32 envs). This is consistent with a per-env workspace that only allocates above a small threshold; the workspace is also the likely source of the super-linear iter-time growth in CablePlug.

8 Recommended operating points

Task	Recommended num_envs	Why
Cable	**4,096** (max throughput)	Throughput peak.
Cable	2,048 (balanced)	92% of peak at half the iter time.
Cable	≤1,024 (sample efficient)	Per-env throughput ≥17 sps/env.
CablePlug	**8–16**	Throughput peak; above is regression.

All raw per-run JSON and stdout logs are kept under `/tmp/cable_scaling/results/`.